



Hands-On Introduction to Game Boy Programming

University of Graz
Dipl.-Ing. Dr. Tristan Repolusk
22.06.2026

Today's Plan



00

Setup

- 09:00–09:10
- Link to Repo & Slides
- Installing the Software we Need

01

Intro

- 09:10–09:50
- Motivation: Why Game Boy?
- Hardware & Components
- Game Boy Assembly
- Development Environment
- Short Break

02

Three Rules

- Important!
- Otherwise possibly hardware defects

03

Hands On

- 10:00–11:15
- Short Break
- 11:30–12:50

00

Setup

Workshop Materials



- **Repository:**
<https://github.com/meteoritenhagel/kfu-gameboy-workshop/>
- **Slides (Deutsch):** gameboy-hands-on-DE.pdf
- **Slides (English):** gameboy-hands-on-EN.pdf



Installation



make (GPL-3.0 License): Build management

- Installation (<https://www.gnu.org/software/make/>)
 - Windows:
 - Download and install <https://www.msys2.org/#installation>
 - MSYS2 `pacman -S make`
 - MacOS: `brew install make`
 - Linux:
 - Apt: `sudo apt install make`
 - Arch: https://archlinux.org/packages/core/x86_64/make/
 - Fedora: <https://packages.fedoraproject.org/pkgs/make/make/>
- Testing: Command `make -v`
Should return: e.g., `GNU Make 4.3`

Installation



git (GPL-2.0 License): Version control

○ Installation (<https://git-scm.com/install/>)

- Windows: `MSYS2 pacman -S msys/git`
- MacOS: `brew install git`
- Linux:
 - Apt: `sudo apt install git`
 - Arch: https://archlinux.org/packages/extra/x86_64/git/
 - Fedora: <https://packages.fedoraproject.org/pkgs/git/git/>

○ Testing: Command `git -v`

Should return: e.g., `git version 2.43.0`

○ Clone the repo: `git clone https://github.com/meteoritenhagel/kfu-gameboy-workshop.git`

Installation

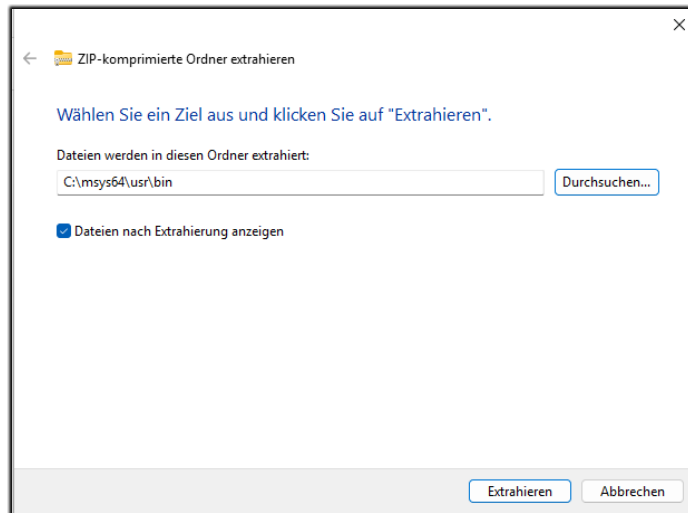


RGBDS (MIT License): Assembler/linker

- Installation (<https://github.com/gbdev/rgbds>)
 - Download the right archive:
github.com/gbdev/rgbds/releases/tag/v1.0.1/
 - Windows
 - Extract to MSYS2 directory `/usr/bin`
(default value: `C:\msys64\usr\bin`)
 - MacOS and Linux:
 - Extract and execute `install.sh`

Testing: Command `rgbasm -V`

Should return: `rgbasm v1.0.1`



Installation



Gearboy (GPL-3.0 License): Emulator/debugger

- Installation (<https://github.com/drhelius/Gearboy/>)
 - Download and extract the correct archive:
github.com/drhelius/Gearboy/releases/tag/3.8.5/
 - Windows:
 - Extract into the MSYS2 directory `/home`
(default value: `C:\msys64\home\<USERNAME>`)
 - `mv /usr/bin/Gearboy.exe /usr/bin/gearboy`
 - MacOS:
 - `ln -s <ABS_PATH>/Gearboy.app/Contents/MacOS/gearboy /usr/local/bin/gearboy`
 - Linux:
 - `ln -s <ABS_PATH>/gearboy /usr/local/bin/gearboy`
 - Testing: Command `gearboy --version`
Should return: `Build: 3.8.5`

Installation



Text Editor, e.g., VSCode (MIT License)

○ Installation (<https://vscodium.com/>)

– Windows: Installation

<https://github.com/VSCodium/vscodium/releases/download/v1.121.03429/vscodium-user-setup-x64-1.121.03429.exe>

– MacOS:

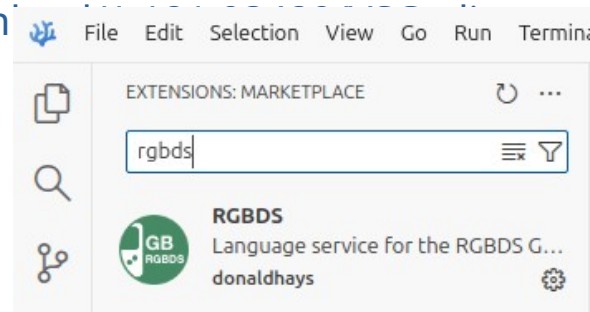
- `brew install --cask vscodium`

– Linux:

- Snap: `sudo snap install codium --classic`
- Aura: `sudo aura -A vscodium-bin`
- Flatpak: `flatpak install flathub com.vscodium.codium`

○ RGBDS Extension (MIT License): <https://github.com/DonaldHays/rgbds-vscode>

– File > Preferences > Extensions



01

Intro

Why Game Boy?



- Simple architecture
- Well-documented
- Many devices supported
- Easy to get real hardware
- Nostalgia

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy (1989)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Super Game Boy (1994)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Pocket (1996)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Color (1998)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Advance (2001)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Player (2003)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



Game Boy Advance SP (2003)

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

Hardware



System on a Chip
Sharp LR35902
(CPU, PPU, APU)

4,19-MHz
Crystal oscillator

8 KB Video RAM

8KB Working RAM

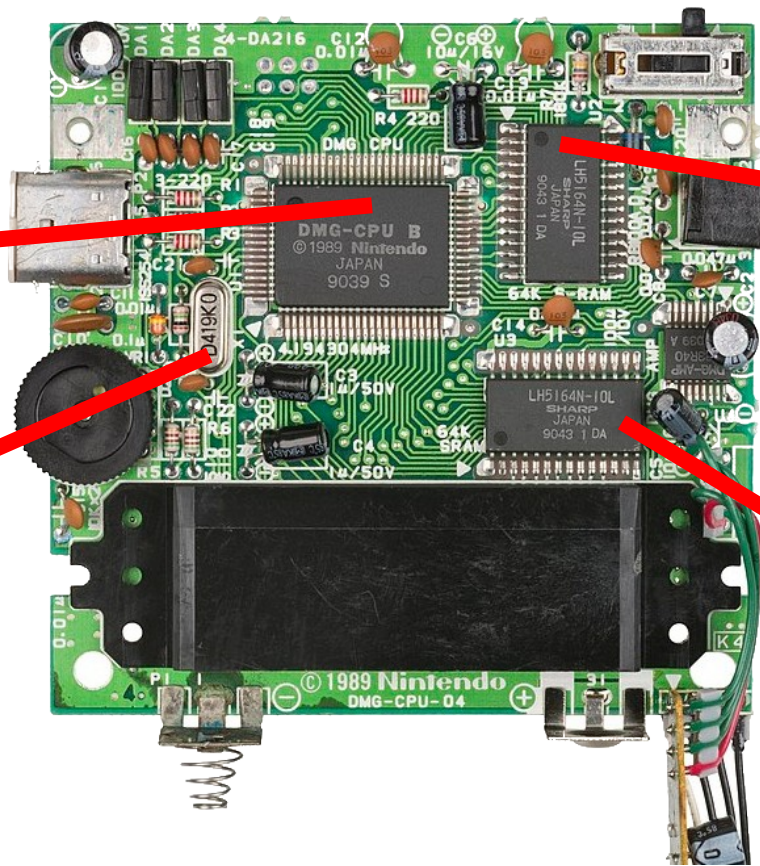


Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

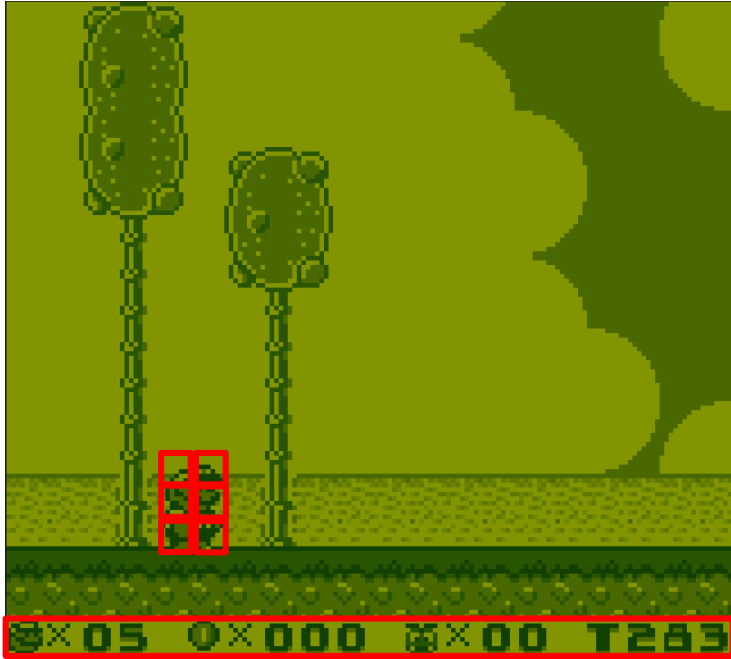
APU (Audio Processing Unit)



	 Duty 1	 Duty 2	 Wave	 Noise
0	D-4 03	D-4 05 CF2	D-4 06 C05	G#7 03
1				
2	D-4 03	A-4 05		
3				
4	A-4 03 0C0	D-4 05		G#7 03
5				
6		A-4 05		
7				
8	D-4 03	D-4 05		G#7 03

- Four different channels
(Square 1, Square 2, Wave, Noise)
- Is directly controlled via CPU instructions (Memory Mapped I/O)
- We use the library hUGEDriver (Public Domain)
<https://github.com/SuperDisk/hUGEDriver>

PPU (Picture Processing Unit)



Source:
Super Mario Land 2. 6 Golden Coins [Nintendo Game Boy]. (1992).
Nintendo.

- Three different layers:
 - Background layer (8x8-Tiles): Environment
 - Window layer (8x8-Tiles, very limited): Status bar
 - Object Layer (8x8 bzw. 8x16 Pixel, pixel-exact position): Projectiles
- Today, we focus on the background layer

PPU (Picture Processing Unit)



- 8 KB VRAM
- Graphics are stored in the 2bpp format
 - 2 bits per pixel → Values 0, 1, 2, and 3
 - 1 tile = 8x8 Pixel → 128 Bits = 16 Bytes



CPU (Central Processing Unit)



▼ Processor X

Z = 0N = 1

H = 0C = 1

A = \$8F

F = \$50

1000 11110101 0000

B = \$00

C = \$00

0000 00000000 0000

D = \$21

E = \$1A

0010 00010001 1010

H = \$9C

L = \$00

1001 11000000 0000

SP = \$FFF8

1111 1111 1111 1000

PC = \$2AB6

0010 1010 1011 0110

- Sharp SM83
- 16-bit address space
- Inspired by Intel 8080 and Zilog Z80
- Registers: af (accumulator + flags), bc, de, hl
- Flags: z, n, h, c
(zero, subtraction, half-carry, carry)
- Stack Pointer (SP) and Program Counter (PC)
- Memory Mapped I/O

Memory Map



**ROM
00**

**\$0000-
\$3FFF**

**ROM
01-NN**

**\$4000-
\$7FFF**

VRAM

**\$8000-
\$9FFF**

(S)RAM

**\$A000-
\$BFFF**

WRAM

**\$C000-
\$DFFF**

**Echo
WRAM**
**\$E000-
\$FDFF**

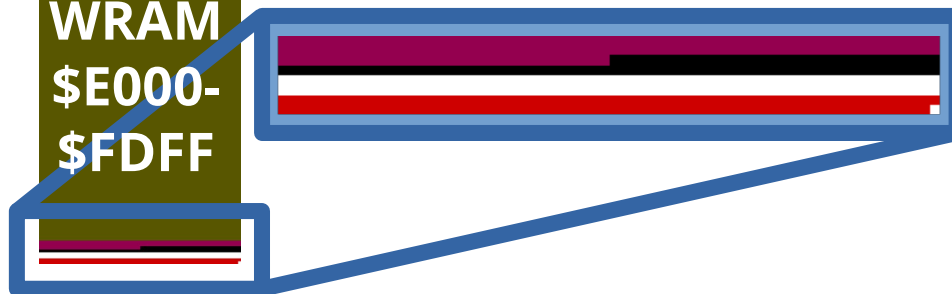
OAM (\$FE00-\$FE9F)

Unusable (\$FEA0-\$FEFF)

I/O (\$FF00-\$FF7F)

High RAM (\$FF80-\$FFFE)

Interrupt Enable (\$FFFF)



Memory Editor

File

Edit

Selection

Bookmarks

Watches

Search

BANKS:

ROM1 \$01

RAM \$00

WRAM1 \$01

VRAM \$00

ROM0

ROM1

VRAM

RAM

WRAM

OAM

IO

HIRAM

ADDR	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
0000:	23	2F	30	39	00	34	28	25	00	32	28	39	34	28	2D	01	#/09.4(%.
0010:	FF	39	2F	35	FF	39	2F	35	00	37	2F	2E	01	FF	00	00	.9/5.9/5.7/.
0020:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0030:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0040:	C3	E6	00	AF	EA	64	C0	21	45	18	CD	50	09	3E	01	EA	...d.!E..P.>..
0050:	64	C0	CD	AE	2A	AF	EA	40	FF	11	50	01	21	00	90	01	d...*...@..P.!...
0060:	00	08	CD	76	2A	11	BF	1D	21	00	98	01	00	04	CD	76	...v*...!.....v
0070:	2A	3E	81	EA	40	FF	3E	FF	EA	47	FF	CD	3E	2B	1E	1E	*>...@.>...G...>+..

0000

GoTo

00

Find Next

REGION: 0000-3FFF

SELECTION: 0048

Dec: 69 (Uint8)

Hex: 45

Bin: 0100 0101

Options

Assembly



○ Machine code:

- Instruction = Opcode + Data,
- Example: GB-Z80, load the value 255 into register **a**:

`00111110 11111111` resp. `0x3E 0xFF`

○ Assembly languages:

- One-to-one correspondence between assembly instructions and machine instructions
- Instruction = Mnemonic + Data
- Example: RGBASM, load the value 255 into register **a**:

`ld a, $FF`

Assembly



- High-level programming languages (z. B. C, C++, Python, ...):
 - Higher level of abstraction
 - Easier to understand
 - Example (C):
Copy the first `n` bytes from `source` to `destination`

```
void memcpy(void *destination, const void *source, size_t n) {  
    uint8_t *dst = destination;  
    const uint8_t *src = source;  
    for (size_t i = 0; i < n; i++) {  
        dst[i] = src[i];  
    }  
}
```

C

```

void memcpy(void *destination, const void *source, size_t n) {
    uint8_t *dst = destination;
    const uint8_t *src = source;
    for (size_t i = 0; i < n; i++) {
        dst[i] = src[i];
    }
}

```

x86-64
ASM

```

; rsi: source
; rdi: destination
; rdx: n
memcpy:
    test rdx, rdx
    je .L1
    mov eax, 0

.L3:
    movzx ecx, BYTE PTR [rsi+rax]
    mov BYTE PTR [rdi+rax], cl
    add rax, 1
    cmp rdx, rax
    jne .L3

.L1:
    ret

```

GB-Z80
RGBASM

```

; de: source
; hl: destination
; bc: n
; only call with bc > 0
; destroys a
memcpy::
    ld a, [de]
    ld [hli], a
    inc de
    dec bc
    ld a, b
    or a, c
    jr nz, memcpy
    ret

```

Most Important Instructions



○ Load instructions:

- `ld a, $64`: $a \leftarrow \$64$
- `ld a, b`: $a \leftarrow b$ (any other combinations of registers possible)
- `ld [$ABCD], a`: Load **a** into the byte with address \$ABCD
- `ld a, [$ABCD]`: Load the byte with address \$ABCD into **a**.
- ...

○ Logical operators with register **a**

- `add a, b` (resp. `add b`): $a \leftarrow a + b$
- Instead of **b** also possible: **a, c, d, e, h, l, [hl]**
- Instead of **and** also the operations **or** and **xor**

Most Important Instructions



- No If/While/For!
- But we have jumps:
 - Jump: `jp label`
 - Relative jump: `jr label`
 - Call subroutine: `call label`
 - Return from subroutine: `ret`, `reti`
- Conditional jumps (Zero Flag, Carry Flag):
 - Jump if zero: `jp z, label`; jump if not zero: `jp nz, label`
 - Jump if carry: `jp c, label`; jump if not carry: `jp nc, label`
 - Similarly for `jp`, `call`, and `ret`



Image: Public Domain by
<https://commons.wikimedia.org/wiki/User:Ocdp>

Control Structures

We simulate if-statements:



CheckEqual::

```
ld a, [wVariable]
cp 5                ; if [wVariable] == 5
call z, VariableEqualFive ; call subroutine VariableEqualFive
ret
```

CheckLess::

```
ld a, [wVariable]
cp 5                ; if [wVariable] < 5
call c, VariableLessFive ; call subroutine VariableLessFive
ret
```

CheckGreaterOrEqual::

```
ld a, [wVariable]
cp 5                ; if [wVariable] >= 5
call nc, VariableGreaterOrEqualFive ; call subroutine GreaterOrEqualFive
ret
```

Control Structures

We simulate for-statements:



```
ForLoop::  
    ld a, 10          ; iterate 10 times  
.loop  
    ; do something  
    dec a  
    jr nz, .loop  
    ret
```

Control Structures

We simulate case-statements:



```
CaseStatement::
    ld a, [wVariable]
    cp 0                ; in case [wVariable] == 0
    jr nz, .notZero
    call VariableIsZero ; call subroutine VariableIsZero
    jr .done

.notZero
    cp 1                ; in case [wVariable] == 1
    jr nz, .notOne
    call VariableIsOne  ; call subroutine VariableIsOne
    jr .done

.notOne
    cp 2                ; in case [wVariable] == 2
    jr nz, .done
    call VariableIsTwo  ; call subroutine VariableIsTwo

.done
    ret ; or other exit, like jp, jr, ...
; ...
```



Short Break

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

02

Three Rules

Three Rules



1) Always power off the display before accessing VRAM



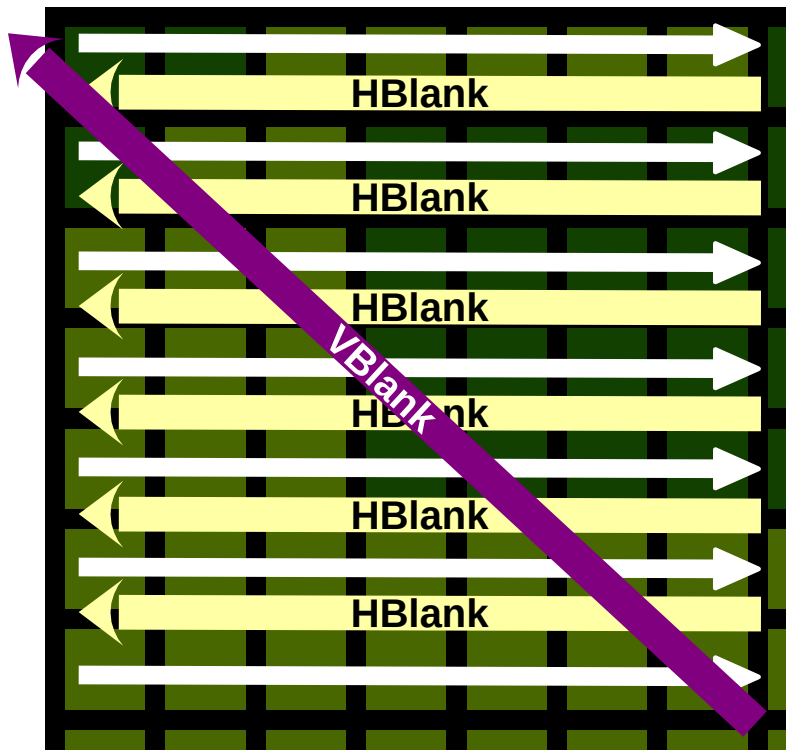
2) **Power off display during Vblank only!**
(Otherwise, hardware damage possible!)



3) **Infinite loops instead of runaway code**



VBlank and HBlank



- HBlank
 - Jump from the end of a line to the beginning of the next line
- VBlank
 - Jump from the end of the last line to the beginning of the first line
- Resolution: 160x144 pixel
 - Draw 144 lines
 - 10 lines VBlank (ca. 1.09 ms)

03

Hands On

Empty Project



```
git checkout 00-empty-project
```

- main.asm
 - ROM Header
 - Entry Point
 - Busy Waiting
- utils.asm
 - Memcopy
 - UpdateKeys

EX1: Display graphics



Prerequisite: `git checkout 00-empty-project`

Goal: Show the title screen

- Copy title.png from raw_resources to project/res
- utils.asm: Write a function WaitVBlank
- Create a new file state_title.asm
 - Write a function InitStateTitle
 - WaitVBlank + power off LCD
 - Include tiles (\$9000) and tilemap (\$9800)
 - Copy tiles, then copy tilemap
 - Power on LCD and initialize palette
- main.asm: Call InitStateTitle



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/00-empty-project...01-title-screen-display>



Short Break

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

EX2: Let the text flash!



Prerequisite: `git checkout 01-title-screen-display`

Goal: Let “Press Start to Play” flash

- `utils.asm`: Write a function `WaitMultipleVBlank`, that waits a certain number of VBlanks
- `state_title.asm`: Change the palettes after half a second each to let the text flash



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/01-title-screen-display...02-title-screen-change-palette>

EX3: React on Player Input



Prerequisite:

`git checkout 02-title-screen-change-palette`

Goal: Leave the title screen when a button is pressed

state_title.asm

- New function WaitMultipleVBlankPressStart that checks for player input every frame
- If input was registered, leave the title screen trough ret



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/02-title-screen-change-palette...03-title-screen-react-on-player-input>

EX4: Add Music



Prerequisite:

`git checkout 03-title-screen-react-on-player-input`

Goal: Let the Music Play

- Move from raw_resources into the project:
 - hUGEDriver/hUGE.inc → include
 - hUGEDriver/hUGE_note_table.inc → include
 - hUGEDriver/hUGEDriver.asm → src
 - huge_music/music_title.asm → src
- variables.asm: wUpdateSound
- start_title.asm: Load the music piece
- main.asm: Initialize + VBlank Interrupt Handler



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/03-title-screen-react-on-player-input..04-title-screen-add-music>



Flash Cart

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>



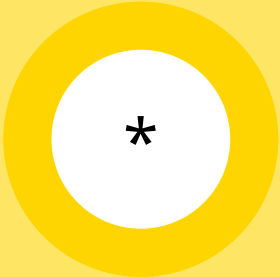
Demo am Emulator

Image: Public Domain by <https://commons.wikimedia.org/wiki/User:Evan-Amos>

Thanks for your participation.

University of Graz
Department of Digital Humanities
Dipl.-Ing. Dr. Tristan Repolusk
Elisabethstraße 59/III, 8010 Graz, Österreich
tristan.repolusk@uni-graz.at
digital-humanities.uni-graz.at



A large yellow circle is positioned on the left side of the slide. Inside this circle is a smaller white circle, which contains a black asterisk (*).

*

Bonus

EX5: Multiple Game States



Prerequisite:

`git checkout 04-title-screen-add-music`

Goal: After button press go into the game screen

- constants.inc: Add two constants for the two game states Title Screen and Game
- variables.asm: wNextState
- New File: state_game.asm
 - Function InitStateGame
- main.asm: Instead of .loop we branch (wNextState) → either to InitStateTitle or InitStateGame
- title.asm: Before ret, we load the next game state into the variable wNextState



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/04-title-screen-add-music...05-switch-between-game-states>

EX6: Graphics of the Game Mechanic



Prerequisite:

`git checkout 05-switch-between-game-states`

Goal: Display the graphics for the game mechanic

- Move game.png from raw_resources to res
- utils.asm: New function StopSounds which mutes all four sound channels
- state_game.asm:
 - Mute sound channels
 - Include and display graphics (cf. state_title.asm)



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/05-switch-between-game-states...06-gameplay-display>

EX7: Display Text



Prerequisite:

```
git checkout git checkout 06-gameplay-display
```

Goal: Display texts for the game mechanic

- Move from raw_resources to project:
 - images/charmap.inc → include
 - images/font.png → res
- utils.asm: New function DrawText which draws text on screen
- state_game.asm:
 - Create text data
 - Load text tiles (\$8800)
 - Find starting position of the texts
 - Call DrawText



<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/06-gameplay-display...07-draw-text>

EX8: Metronome (Part 1)



Prerequisite:

`git checkout 07-draw-text`

Goal: Play a beat noise

- Copy `huge_music/music_weakbeat.asm` → `src`
- `utils.asm`:
 - New function `PlayWeakBeat`
 - Plays a weak beat noise (in `music_weakbeat.asm`)
 - Push all registers
 - Load the sound (cf. `state_title.asm`)
 - Pop all registers



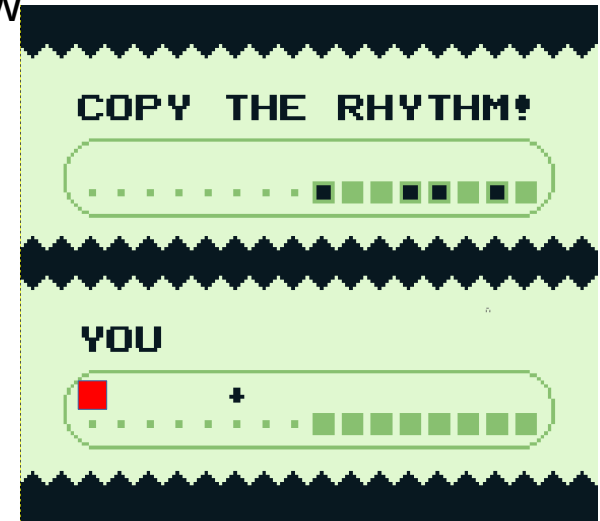
<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/07-draw-text...08-gameplay-metronome>

EX8: Metronome (Part 2)



Goal: Implement the metronome for the game mechanic

- Find the starting position (marked red)
- Find the tile numbers of the background and arrow
- state_game.asm:
 - Load starting position into hl
 - Write the function PlayMetronome:
 - PlayWeakBeat
 - Change tile from background to arrow
 - Wait for some time
 - Change tile back to background
 - In total for all eight tiles!



EX9: Player Input in the Game (Part 1)



Prerequisite:

`git checkout 08-gameplay-metronome`

Goal: Play a stronger beat noise

- `huge_music/music_strongbeat.asm` → ?
- `utils.asm`: Write `PlayStrongBeat`



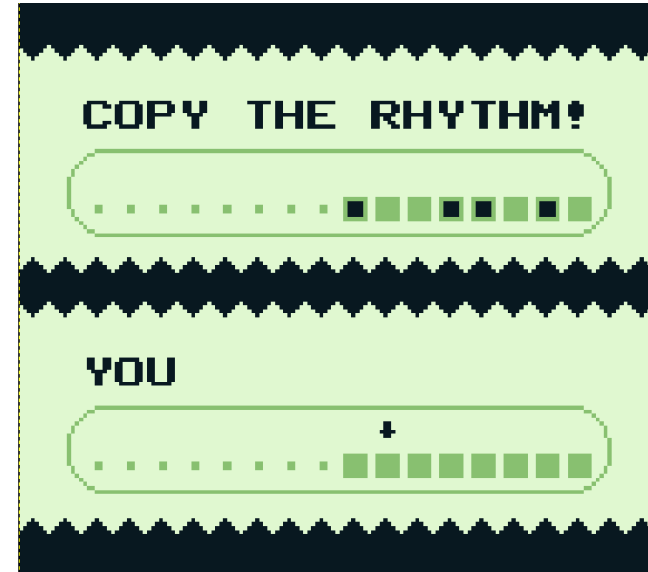
<https://github.com/meteoritenhage/kfu-gameboy-workshop/compare/08-gameplay-metronome...09-gameplay-react-on-player-input>

EX9: Player Input in the Game (Part 2)



Goal: Player Input → Box is marked as black

- Find tile numbers of empty and full box
- state_game.asm:
 - New function PlayerInput as a copy of PlayMetronome
 - Instead of WaitMultipleVBlank, write a new function WaitMultipleVBlankPlayerInput (cf. state_title.asm)
 - Check for player input:
 - if yes: PlayStrongBeat and change box to filled (adding \$20 to the address means going one tile to the bottom)
 - if no: Continue normally



More Exercises



- **10-gameplay-clear-boxes:**

More rounds of game play with resetting the boxes after each round

- **11-gameplay-given-beat-pattern:**

Pseudo-random new beat patterns for each game play round

- **12-gameplay-store-player-input:**

Player input is stored in a variable

- **13-gameplay-compare-given-and-player-inputs:**

We compare player input beat pattern and ground truth beat pattern

More Exercises



- **14-gameplay-change-tempo-each-round:**

Increase tempo after each successful round

- **15-gameplay-add-success-text:**

If the game is played through, display the text "YOU WON!"

- **16-nice-screen-transitions:**

Make extra fancy transitions between different screens for nice visuals